Final project - Drum Kit Classifier

Abstract:

The purpose of this project was to see if machine learning models can be trained to classify drum sounds by their formant and timbre. The goal is to train a model that can classify a drum hit into 1 of 4 drum sounds. These drum sounds are crash cymbal, high-hat, kick drum, and snare drum.

Dataset:

The dataset used for this project was found on Kaggle with the title “Drums-wavegan” and posted by user weiteng hu.

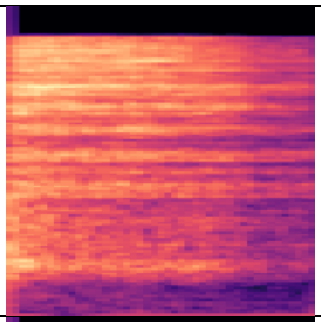
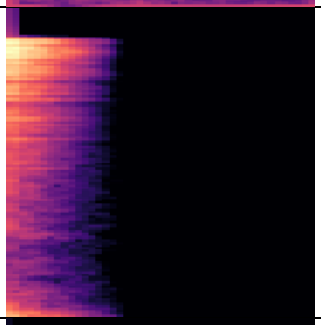
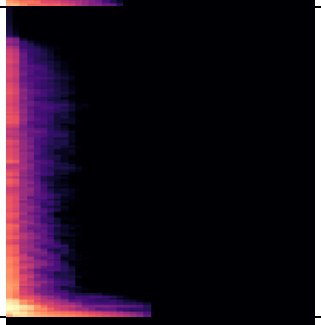
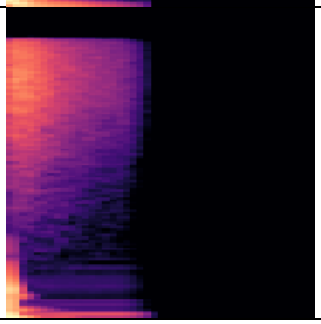
The dataset originally contained more classes of drums than the 4 that I wanted to use for this project. I filtered out the unwanted classes and sorted the files into folders for each class

Classes from the dataset used (Labels):

1. Crash
2. Hat
3. Kick
4. Snare

Once sorted, I split the data into an 80%, 20% split for training and testing. The training set was 1525 records, the testing set 381 records, meaning the total records used were 1,906.

The dataset was originally audio .wav files. I turned them into Mel spectrograms before fitting the models.

1. Crash	
2. Hat	
3. Kick	
4. Snare	

Models:

For the first model, I normalized and reduced the dimensions of the data using Standard Scaler and PCA (Principal Component Analysis) with components set to 200. I then trained the SVC model.

SVC (Support Vector Classification)

Kernel	poly
Degree	2

Coef0	1.0
Probability	False

For the second model, I used a CNN (convolutional neural network).

Model variables

Optimizer	Adam optimizer
Scheduler	ExponentialLR
Batch size	32
Iterations	100
Learning rate	0.00002
Gamma	0.975

Model layout

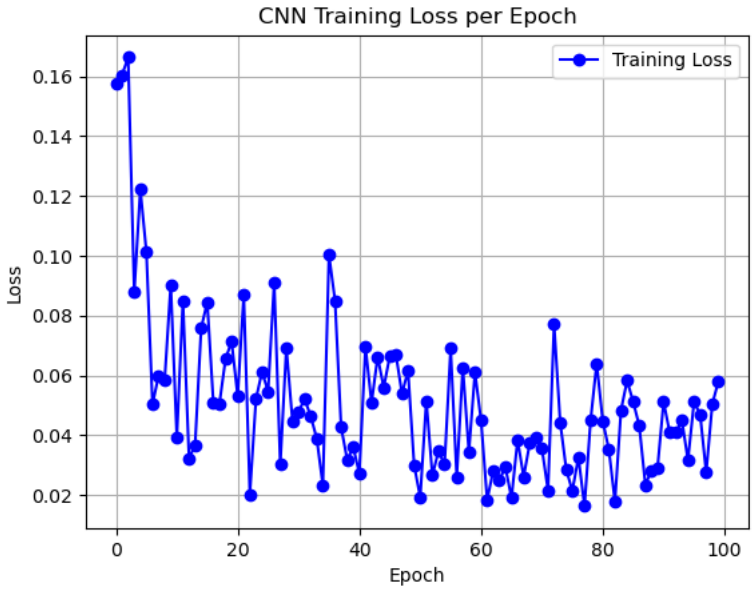
```
model1 = nn.Sequential(
  nn.Conv2d(in_channels=3, out_channels=channels, kernel_size=ksize),
  nn.ReLU(),
  nn.MaxPool2d(2, 2),
  nn.Conv2d(in_channels=channels, out_channels=channels * 2, kernel_size=ksize),
  nn.ReLU(),
  nn.MaxPool2d(2, 2),
  nn.Flatten(),
  nn.Linear(K3, 128),
  nn.ReLU(),
  nn.Linear(128, class_size),
)
```

Results:

	CNN	SVC
Error Rate	7.09%	6.04%
Wrong count	27	23
Test size	381	381

Training time	40 minutes	1 minute

CNN training loss



Code used:

.wav to spectrogram

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import os
import numpy as np
num = "vald"
input_folder = os.path.join("drums", "valid")
output_folder = num

os.makedirs(output_folder, exist_ok=True)

for fname in os.listdir(input_folder):
    if fname.endswith(".wav"):
        file_path = os.path.join(input_folder, fname)
```

```
        # Load audio
        y, sr = librosa.load(file_path, sr=22050)   # standard sampling
rate

        # Compute Mel spectrogram
        mel = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=2048,
                                             hop_length=512, n_mels=128)
        mel_db = librosa.power_to_db(mel, ref=np.max)

        # Save as .png
        plt.figure(figsize=(3, 3))
        librosa.display.specshow(mel_db, sr=sr, hop_length=512,
x_axis=None, y_axis=None)
        plt.axis("off")

        out_path = os.path.join(output_folder, fname.replace(".wav",
".png"))
        plt.savefig(out_path, bbox_inches="tight", pad_inches=0)
        plt.close()
```

SVM model

```
import os
import numpy as np
from PIL import Image
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC

def load_images(dataset_folder):
    data = []
    classification = []
    class_labels = ['1', '2', '3', '4']
    for label in class_labels:  # iterate over folder names
        label_folder = os.path.join(dataset_folder, label)
        if not os.path.isdir(label_folder):
            continue  # skip if not a folder
```

```
        for file in os.listdir(label_folder):
            if file.startswith('.'):
                continue  # skip hidden files like .DS_Store
            image_path = os.path.join(label_folder, file)
            try:
                image = Image.open(image_path).convert('RGB')
            except Exception as e:
                print(f"Skipping file {image_path}: {e}")
                continue
            img_as_arr = np.array(image, dtype=np.float32) / 255.0
            data.append(img_as_arr.flatten())
            classification.append(label)
    return data, classification


train_data, train_labels = load_images("drums_train")
test_data, test_labels = load_images("drums_test")

# StandardScaler
scaler = StandardScaler()
train_data = scaler.fit_transform(train_data)
test_data  = scaler.transform(test_data)

# PCA
pca = PCA(n_components=200)
train_data = pca.fit_transform(train_data)
test_data  = pca.transform(test_data)

# Train SVC
svc_model = SVC(kernel='poly',degree=2,coef0=1.0,probability=False)
svc_model.fit(train_data, train_labels)

# Test
y_hat = svc_model.predict(test_data)
wrong = np.sum(y_hat != test_labels)
count = len(test_labels)
print('count:', count)
print('wrong:', wrong)
print(f"Percent Error: {100 * wrong / count}%")
```

CNN model

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torchvision
from torchvision.transforms import v2
from PIL import Image

train_image_dir = "drums_train"
test_image_dir = "drums_test"

class_size = 4
batch_size = 32
MAX_ITER = 100
LR = 0.00002
gamma = 0.975

#color image
img_transform = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32,
scale=True)])

#grayscale image
#img_transform =
v2.Compose([v2.ToImage(),v2.ConvertImageDtype(torch.float32),v2.Grayscale(
num_output_channels=1)])

train_dataset = torchvision.datasets.ImageFolder(
    train_image_dir, transform=img_transform
)

labels = train_dataset.classes
label_count = len(labels)
print(labels)

train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=batch_size, shuffle=True
)
```

```
if False:
    data_iterator = iter(train_loader)
    X, Y = next(data_iterator)

    image_grid = torchvision.utils.make_grid(X, padding=5, nrow=8)
    plt.imshow(np.transpose(image_grid.numpy(), (1, 2, 0)))
    plt.axis("off")
    plt.show()
    exit()


channels = 16
ksize = 3
K3 = (channels * 2) * 56 * 56  # Flattened size after second pooling layer

model1 = nn.Sequential(
    nn.Conv2d(in_channels=3, out_channels=channels, kernel_size=ksize),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Conv2d(in_channels=channels, out_channels=channels * 2,
kernel_size=ksize),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Flatten(),
    nn.Linear(K3, 128),
    nn.ReLU(),
    nn.Linear(128, class_size),
)


model = model1
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=gamma)
loss_func = nn.MSELoss()
loss_values = []
for epoch in range(MAX_ITER):
    for batch in train_loader:
        x_data, y_data = batch
        y_data = nn.functional.one_hot(y_data,
num_classes=label_count).type(
```

```
            torch.float32
        )
        y_hat = model(x_data)
        loss = loss_func(y_hat, y_data)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(epoch, loss.item())
    loss_values.append(loss.detach().numpy())
    scheduler.step()

test_dataset = torchvision.datasets.ImageFolder(test_image_dir,
transform=img_transform)

test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=batch_size, shuffle=True
)

count = 0
wrong = 0

data_iterator = iter(test_loader)
for X, Y in data_iterator:
    y_hat = model(X)
    y_hat = torch.argmax(y_hat, axis=1)
    Z = (Y - y_hat).detach().numpy()
    print(Z)
    count += len(Z)
    wrong += np.count_nonzero(Z)


print("Count:", count)
print("Wrong:", wrong)
print("Percent Error:", 100 * wrong / count,'%')

epochs = list(range(0, MAX_ITER))
plt.plot(epochs, loss_values, color='blue', marker='o', label='Training
Loss')
plt.title('CNN Training Loss per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
plt.grid(True)
plt.legend()
plt.show()
# Save the trained model
model_save_path = "/Users/tylerdurand/Downloads/School
Stuff/math3375/final_model1_batch32.pth"
torch.save(model.state_dict(), model_save_path)
print(f"Model saved to {model_save_path}")
```

CNN model demo app

```
import streamlit as st
import torch
import torch.nn as nn
from torchvision.transforms import v2
from PIL import Image
import librosa
import librosa.display
import matplotlib.pyplot as plt
import os
import numpy as np

# Define the model
def create_model():
    channels = 16
    ksize = 3
    K3 = (channels * 2) * 56 * 56  # Flattened size after second pooling
layer

    model = nn.Sequential(
        nn.Conv2d(in_channels=3, out_channels=channels,
kernel_size=ksize),
        nn.ReLU(),
        nn.MaxPool2d(2, 2),
        nn.Conv2d(in_channels=channels, out_channels=channels * 2,
kernel_size=ksize),
        nn.ReLU(),
        nn.MaxPool2d(2, 2),
        nn.Flatten(),
```

```
        nn.Linear(K3, 128),
        nn.ReLU(),
        nn.Linear(128, 4),
    )
    return model

# Classify the uploaded image
def classify_image(image, model, labels):
    transform = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32,
scale=True)])
    image_tensor = transform(image).unsqueeze(0)
    transform = v2.Compose([
        v2.ToImage(),
        v2.Resize((224, 224)),  # Resize to match model input size
        v2.ToTensor(),
        v2.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])  # Normalize for pretrained models
    ])
    # Perform classification
    with torch.no_grad():
        output = model(image_tensor)
        predicted_class = torch.argmax(output, axis=1).item()

    return labels[predicted_class]

def spectrogram_conversion(wav_file):

        # Load audio
        y, sr = librosa.load(wav_file, sr=22050)   # standard sampling
rate

        # Compute Mel spectrogram
        mel = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=2048,
                                             hop_length=512, n_mels=128)
        mel_db = librosa.power_to_db(mel, ref=np.max)

        # Save as .png
        plt.figure(figsize=(3, 3))
        librosa.display.specshow(mel_db, sr=sr, hop_length=512,
x_axis=None, y_axis=None)
        plt.axis("off")
```

```
        out_path = "/Users/tylerdurand/Downloads/School
Stuff/math3375/temp_spectrogram.png"
        plt.savefig(out_path, bbox_inches="tight", pad_inches=0)
        plt.close()


# Streamlit app
def main():
    st.title("Drum Sound Classifier")
    st.write("Upload an image to classify it.")

    model_path = "/Users/tylerdurand/Downloads/School
Stuff/math3375/final_model1.pth"
    labels = ["Crash","Hat","Kick","Snare"]

    uploaded_file = st.file_uploader("Upload Spectrogram",
type=["png","wav"])

    if uploaded_file is not None:

        spectrogram_conversion(uploaded_file)
        # Display the uploaded image
        image = Image.open("temp_spectrogram.png").convert("RGB")
        st.image(image, caption="Uploaded Image", use_column_width=True)

        # Load the model
        model = create_model()
        state_dict = torch.load(model_path,
map_location=torch.device('cpu'))
        model.load_state_dict(state_dict)
        model.eval()

        # Classify the image
        predicted_label = classify_image(image, model, labels)
        st.write(f"Predicted Label: **{predicted_label}**")

if __name__ == "__main__":
    main()
```